Carleton University
School of Computer Science

COMP 3000 (WINTER 2023) OPERATING SYSTEMS ASSIGNMENT 1

Please submit the answers to the following questions in Brightspace by the due time indicated in the submission entry. There are 20 points in total (weight: 0.25).

Submit your answers as a **gzipped tarball** "*username*-comp3000-assign1.tar.gz" (where username is your MyCarletonOne username). Do NOT just submit a file of another format (e.g., txt or docx) by renaming it to .tar.gz. Unlike tutorials, assignments are graded for the correctness of the answers.

The tarball you submit **must** contain the following (applicable to all assignments):

1. A **plaintext** file containing your solutions to all questions, including explanations.
2. A README.txt file listing the contents of your submission as well as any information the TAs should know when grading your assignment. Without this file, grading will be based on the TA's understanding.
3. For each question, where applicable, a C file for your modified version of the original file. This should include all required changes for that question. This way, you can avoid something wrong with one question affecting another question.
4. Diff files showing the modifications, by comparing each submitted C file above and the original: for example, `diff -c 3000original.c 3000modified.c > 3000.diff`. Avoid moving around or changing existing code (unless necessary) which may be distracting.

For this assignment, you only need to submit code for question 5c in part 2. Other answers should remain in your plaintext file.

You can use this command to create the tarball: `tar zcvf username-comp3000-assign1.tar.gz your_assignment_directory`. ****Don't forget to include your plaintext file!!****

No other formats will be accepted. Submitting in another format will likely result in your assignment not being graded and you receiving no marks for this assignment. In particular, do not submit an MS Word, OpenOffice, or PDF file as your answers document!

As a student learning operating system, you are expected to be able to handle file transfer properly using necessary commands such as `ssh`, `scp` or `rsync`. Otherwise (which happens), **please set aside time by completing your assignment enough long before the due time**, then you may ask a TA, the instructor, or another student for any file transfer issue. Then, you can keep working on it and **override your submission** by uploading a new tarball before it is due.

For your convenience, here are a few commands (among many others) to download a file from your Openstack VM. You need to run the command on your PC, not your Openstack VM.

Empty or corrupted tarballs may be given a grade of zero, so please **double check your submission** by downloading and extracting it.

Don't forget to include what outside resources you used to complete each of your answers, including other students, and web resources. You do not need to list help from the instructor, TA, or information found in the textbook or man pages.

Use of any outside resources verbatim as your answer (like copy-paste or quotation) is not allowed, and will be treated as unauthorized collaboration (and reported as plagiarism).

Please do **NOT** post assignment solutions on MS Teams or Brightspace (or other platforms not used in the course such as Discord) or it will be penalized. Moreover, posting the assignment questions to any external forums/websites is NOT permitted and will also be penalized/reported.

Questions – part 1 [6]

The `sudo` command is a convenient way of executing commands as the root user (by default, unless another user is specified) if the current user is allowed to (by `/etc/sudoers`). Now, answer the following questions.

1. [2] To set an environment variable, you can "`export MYVAR=99`", then you will be able to see it using the command "`env`". But if you "`sudo env`" (running `env` as the root user) you won't see it. Why [1/2]? What simple argument can you add to "`sudo env`" to see "`MYVAR=99`" [1/2]?
2. [2] You have just learned the concepts of user/kernel modes (hardware-enforced) and root/non-root users (OS-enforced), which are orthogonal to each other. Briefly describe when different mode-user combinations happen during the execution of "`sudo env`". As an example of one combination, when it is doing X, it runs in kernel mode as the regular user (e.g., student).
3. [1] Explain why "`sudo echo $MYVAR`" works as is, without the fix above in question 1?
4. [1] Why does "`sudo export MYVAR=99`" fail?

Questions – part 2 [7]

The following questions (where applicable) will be based on the original 3000memview.c in Tutorial 2:

5. When you change each `malloc()` call to allocate more than 128KiB, you may have noticed (or will notice) that `malloc()` switches between the system calls `mmap()` and `brk()` for memory allocation depending on a preset threshold (which in this case by default is 128KiB). With the help of **`man mallopt`**, answer the following questions. Note that this trains you to be able to search for a solution by skimming through available documentation (so no need to read it word by word).
    a. [2] What are the two ways to change this threshold? Be specific.
    b. [2] To change this threshold to 200KiB (i.e., 200 x 1024) using one of the two ways, provide the command(s) as your answer.
    c. [3] Modify `3000memview.c` to implement the other way. Your code must compile.

    For both b and c, you (or the TA) should be able to verify the threshold change by replacing `malloc(4096)` with `malloc(199*1024)` and `malloc(200*1024)` respectively in your `3000memview.c`, and watch the `strace` output.

Questions – part 3 [7]

The following questions (where applicable) will be based on the original hello.c in Tutorial 1, syscall-hello.c in Tutorial 2, and this new syscall2-hello.c. Compile and run them.

6. All the three programs do the same thing, i.e., print "Hello world!" as the output. But speaking of portability (as we discussed in class), they are different.

a. [2] Order the three programs from the least portable to the most portable (just list them in order as your answer).
b. [3] Explain why you ordered them this way (again, be specific for each program).

7. [2] What is the key difference between `syscall-hello.c` and `syscall2-hello.c`, in terms of types of calls made for the same purpose? Without relying on any assembly knowledge, how do you know? (using only commands we have used so far)